

Urban Tree Coverage: um pipeline ponta a ponta reproduzível para cobertura arbórea visível em imagens de rua

Juan Oliveira de Carvalho¹

¹Universidade Federal do ABC (UFABC) – Santo André – SP – Brasil

juano.oliveira@aluno.ufabc.edu.br

Abstract. *Urban Tree Coverage is a working tool for measuring visible tree coverage in street-level imagery. One core takes a local image, a coordinate pair or an address, fetches and caches Street View panoramas, plans single or multi-view captures, runs one of four segmentation backends behind a common interface, maps model classes to tree and vegetation groups through an editable taxonomy, and writes metrics, masks and a run manifest. A CLI and a Web API share that core. To check it, seven frames were annotated by hand (41 trees): OneFormer stayed within 0.963 percentage points of the manual coverage. The pilot spans three locations, so it validates the instrument rather than ranking architectures; it also shows that changing the label space alone removes the indicator.*

Resumo. *O Urban Tree Coverage é uma ferramenta para medir cobertura arbórea visível em imagens de rua. Um mesmo núcleo recebe imagem local, coordenada ou endereço, obtém panoramas do Street View com cache, planeja capturas em vista única ou múltipla, executa um entre quatro backends de segmentação sob a mesma interface, mapeia classes para grupos de árvore e vegetação por taxonomia editável e grava métricas, máscaras e manifesto. CLI e Web API compartilham esse núcleo. Para verificá-lo, sete quadros foram rotulados à mão (41 árvores): o OneFormer ficou a 0,963 ponto percentual da cobertura manual. O piloto cobre três localizações, valida o instrumento e não ranqueia arquiteturas; mostra, ainda, que trocar o espaço de classes com a arquitetura fixa retira o indicador.*

1. Descrição do Problema

1.1. Contexto e motivação

Observar arborização em escala de cidade é caro. Inventários de campo descrevem indivíduos, espécies e dimensões, com utilidade operacional direta; imagens aéreas estimam a projeção superior das copas. Nenhuma das duas fontes responde ao que uma pessoa enxerga ao caminhar pela calçada. Foi para essa dimensão visual que se formulou o *Green View Index* (GVI), definido como a fração de pixels de vegetação em fotografias tomadas do nível do solo [Li et al. 2015, Seiferling et al. 2017], e a disponibilidade do Google Street View (GSV) tornou a coleta viável em larga escala [Cai et al. 2018].

Boa parte dessa literatura publica o indicador; poucos trabalhos publicam o instrumento. Quem quer repetir um estudo desses acaba reconstruindo tudo: cliente do GSV, cache, escolha de ângulos, adaptação de algum modelo de segmentação, mapeamento de

classes, pós-processamento, agregação e exportação. Cada reconstrução pode alterar qual classe conta como árvore, se o denominador inclui a faixa de atribuição e se o rumo foi escolhido antes ou depois de olhar a máscara.

O objetivo central deste trabalho é o instrumento. O *Urban Tree Coverage* é um pipeline que um usuário final consegue executar do começo ao fim: entra imagem local, par de coordenadas ou endereço; saem cobertura por vista, estatísticas de agregação, máscaras, sobreposições e um manifesto que registra versões, checkpoint, dispositivo e taxonomia. O mesmo núcleo atende a CLI `tree-ai` e uma Web API em FastAPI, opera em vista única (*single-view*) ou em um plano de múltiplas vistas (*multi-view*) e troca de backend por configuração. O código, com documentação em inglês e português mediante uso futuro em pesquisa, está sob licença Apache-2.0 em https://github.com/juanocv/urban_tree_coverage [Carvalho 2026b].

O alvo é deliberadamente estreito: em vez de tratar toda região verde como equivalente, o indicador conta apenas pixels atribuíveis a árvores. Gramados, sebes, arbustos e trepadeiras têm valor no espaço urbano, mas prestam serviços distintos dos da arborização viária e não deveriam entrar no mesmo número sem aviso. Uma medida ampla de vegetação continua disponível, separada, quando o espaço de classes do modelo permite; ela nunca é renomeada como árvore.

A comparação entre backends, cujo protocolo está na Seção 2.6 e cujos números estão na Seção 3, contempla o projeto como verificação, não como fim. Era preciso verificar se o núcleo, alimentado por pesos e taxonomias diferentes, produzia números compatíveis com anotação manual e quanto da variação vinha da escolha de classe em vez da arquitetura. Sete quadros e três localizações bastam para essa checagem. Não bastam para ordenar arquiteturas, e este trabalho não o faz.

As contribuições, nessa ordem de importância, são: um pipeline executável ponta a ponta, com CLI e API sobre o mesmo núcleo, artefatos por execução e testes offline; uma taxonomia inspecionável e substituível, que distingue classe real, proxy declarado e indisponibilidade; planejamento determinístico de vistas, independente da máscara; e um piloto rotulado à mão que serve de verificação do conjunto e expõe o efeito do espaço de classes sobre a medida.

1.2. Definição formal da tarefa

Seja uma imagem RGB $I \in \{0, \dots, 255\}^{H \times W \times 3}$, com altura H e largura W , e seja $\hat{y}_p$ o rótulo previsto para o pixel p . A máscara binária de árvore é $M_T(p) = \mathbb{1}[\hat{y}_p \in \mathcal{T}]$, em que $\mathcal{T}$ reúne as classes que a taxonomia da execução mapeia para o grupo `tree`. O indicador por vista é

$$r_T(I) = \frac{\sum_{p \in I} M_T(p)}{H W}, \quad P_T(I) = 100 r_T(I). \quad (1)$$

No código esses valores aparecem como `tree_coverage_ratio` e `tree_coverage_pct`. O denominador é a imagem inteira entregue ao pipeline, bordas e faixa de atribuição do GSV incluídas. Fixar essa convenção evita que recortes diferentes produzam números com o mesmo nome, ao custo de uma pequena redução na razão. Não há conversão para metros quadrados: sem geometria de câmera, profundidade, orientação

da superfície e tratamento de oclusão, área de pixel não é área métrica.

Contagem também está fora de alcance. Duas copas sobrepostas formam uma região semântica única, e a mesma árvore reaparece em vistas diferentes sem que o sistema saiba disso. O que a Equação 1 estima é a proporção do campo de visão ocupada por árvores. O nome *tree coverage* é prático no software; no texto adota-se *cobertura arbórea visível*, para não confundir com cobertura de copa medida por sensoriamento remoto.

1.3. Trabalhos relacionados e o problema do espaço de classes

O GVI clássico quantifica verde percebido no nível da rua. Li et al. automatizaram com vistas do GSV uma medida antes obtida à mão [Li et al. 2015]; Seiferling et al. mapearam árvores urbanas a partir de imagens de rua [Seiferling et al. 2017]; o Treepedia 2.0 mostrou que segmentação profunda supera métodos não supervisionados nesse cenário [Cai et al. 2018]. Somar toda vegetação responde a uma pergunta legítima, porém diferente: um muro tomado por hera eleva o GVI sem constituir arborização viária, e uma sebe alta pode ser lida como copa em taxonomias grosseiras. Por isso a opção por publicar P_T e, quando a arquitetura permite, uma medida separada P_V para a união declarada de vegetação.

Do lado dos modelos, Mask2Former trata segmentação semântica, de instâncias e panóptica como classificação de máscaras, com atenção restrita às regiões previstas [Cheng et al. 2022]. OneFormer acrescenta condicionamento pela tarefa e treinamento conjunto, executando as três modalidades com um modelo só [Jain et al. 2023]. Panoptic FPN combina ramos de instância e semântico sobre uma pirâmide de características compartilhada [Kirillov et al. 2019a], e o DeepLabV3+ usa convolução *atrous*, pirâmide espacial e um decodificador para recuperar contornos [Chen et al. 2018].

Capacidade arquitetural, porém, não garante classe utilizável. Segmentação panóptica separa classes contáveis (*things*) de regiões amorfas (*stuff*) [Kirillov et al. 2019b]. Nos checkpoints usados neste trabalho, *tree* do ADE20K e *tree-merged* do COCO-panoptic são *stuff*, e no Cityscapes a classe *vegetation* agrega árvores, arbustos e sebes. Um modelo capaz de segmentar instâncias não passa a produzir instâncias de árvore só porque a chamada de pós-processamento mudou; os pesos precisariam ter sido treinados com árvore como *thing*.

2. Abordagem

2.1. Arquitetura do pipeline

O projeto reaproveita a organização modular do repositório autoral *sidewalk_analysis* [Carvalho 2026a]: cliente GSV, cache, geocodificação, configuração por ambiente, injeção de dependências, estrutura de CLI e API e filosofia de testes. Considerando que o alvo atual é adimensional, foram removidos os componentes ligados a calçadas, como profundidade, câmera, conversão pixel-metro, largura, obstáculos, folgas e regras de acessibilidade.

O núcleo é a classe `CanopyPipeline`. Um backend injetado recebe RGB e devolve `SegmentationOutput`: nome do backend, espaço de classes, taxonomia, mapa de rótulos, segmentos e uma máscara booleana por grupo semântico. A saída é validada contra a dimensão da imagem e contra os grupos declarados antes de seguir adiante. Com isso, as diferenças entre bibliotecas ficam confinadas aos adaptadores,

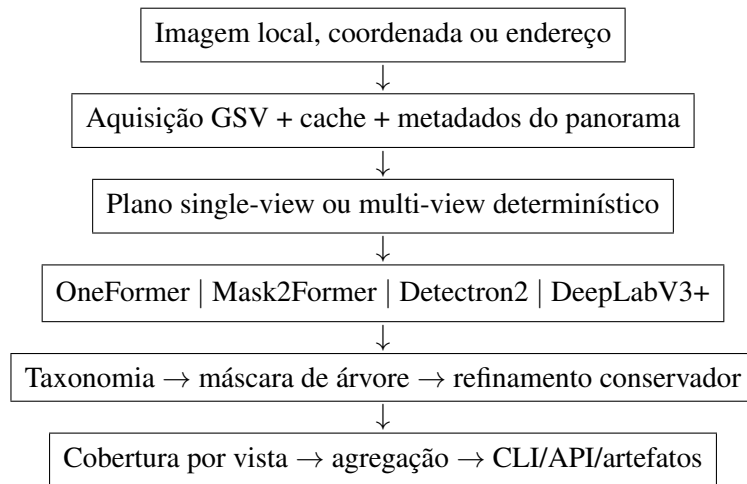


Figura 1. Fluxo funcional. A avaliação contra o COCO manual é uma etapa offline que associa predições e anotações pelo nome original da imagem.

enquanto aquisição, pós-processamento, medida e exportação percorrem sempre o mesmo caminho, que é a condição para comparar dois backends sem comparar, junto, duas implementações inteiras de pipeline.

2.2. Aquisição e planejamento de vistas

Imagens locais são decodificadas em RGB `uint8` sem alterar a resolução. Para o GSV, `StreetViewClient` aceita coordenadas ou endereço e guarda a resposta em cache por posição, rumo, inclinação, campo de visão e tamanho, validando o conteúdo antes da escrita atômica. Uma consulta de metadados por localização registra `pano_id` e data de captura, porque o provedor atualiza imagens e reassocia coordenadas a novos panoramas [Google Maps Platform 2026]; para reprodução estrita, imagem e metadados precisam ser preservados juntos.

O plano de vistas tem três modos: `fixed` recebe angulações explícitas, `offsets` soma deslocamentos a uma orientação de referência e `equiangular` divide 360° em n vistas. A seleção é função pura da configuração e nunca consulta a máscara, pois escolher depois da segmentação a câmera que mostra mais árvores seria viés de seleção embutido no instrumento. Falhas de `download` e de análise são devolvidas por rumo e por etapa (`fetch` ou `analysis`) em vez de desaparecerem no log, e `-min-successful-views` impede que uma execução planejada com oito câmeras se apresente como completa depois de obter duas.

2.3. Backends e taxonomia

A Tabela 1 resume os quatro backends. OneFormer e Mask2Former usaram checkpoints ADE20K, cujo benchmark denso tem 150 classes de objetos e *stuff* [Zhou et al. 2017]. O Detectron2 usou Panoptic FPN R50 no espaço COCO-panoptic. O DeepLabV3+ usou pesos Cityscapes-19; o Cityscapes reúne cenas viárias de 50 cidades, com 5.000 quadros finamente anotados e 20.000 com anotação grosseira, mas suas instâncias cobrem pessoas e veículos, e a classe de vegetação não individualiza árvores [Cordts et al. 2016].

A escolha desses quatro tem origem prática. Detectron2, DeepLabV3+ e OneFormer já estavam integrados ao *sidewalk_analysis*, com adaptadores escritos e dependências

Tabela 1. Backends disponíveis e consequência do espaço de classes para o indicador.

Backend	Pesos/dados	Classe usada	Consequência
OneFormer	ADE20K, Swin-L	tree + palm	Cobertura de árvore disponível; máscaras semânticas.
Mask2Former	ADE20K, COCO ou Cityscapes	depende do checkpoint	Único backend publicado em vários espaços de classes.
Detectron2	COCO-panoptic, R50	tree-merged	Árvore é <i>stuff</i> ; todas as copas formam região sem identidade.
DeepLabV3+	Cityscapes, MobileNet	vegetation	Cobertura vegetal disponível; cobertura de árvore fica indisponível.

resolvidas, e vieram junto com o resto da infraestrutura. O Mask2Former entrou depois porque é distribuído no mesmo repositório de modelos do OneFormer e exige as mesmas dependências, o que tornou o adaptador barato, e, por acaso, deu ao projeto o único backend publicado em vários espaços de classes, do qual a Seção 3.2 tira proveito. Nenhum outro backend foi avaliado e descartado. O conjunto é o que a infraestrutura já suportava, não uma seleção desenhada para cobrir o espaço de arquiteturas.

A classe `Taxonomy` concentra os aliases e a composição dos grupos. No ADE20K, `tree`, `palm` e `palm tree` formam `tree`, `grass` permanece isolado e plantas, flores e arbustos formam `plant_shrub`. No COCO-panoptic, `tree-merged` é árvore. No Cityscapes existem `vegetation` e `terrain`, com `tree_group=None`. Um arquivo JSON substitui a taxonomia inteira, desde que declare o mesmo espaço do checkpoint.

Caso exista algum grupo de árvore, a origem da medida é `tree_class`. Caso contrário, o valor é `None` e `tree_source` recebe `unavailable`. Quando o *back-end* dispõe de vegetação, a *flag* `-allow-vegetation-proxy` permite estimar cobertura de árvore a partir dessa classe; o resultado passa então a carregar `tree_from_vegetation_proxy` e um *quality flag* até a exportação final. No piloto, este *proxy* ficou desligado, de modo que o DeepLab entra na comparação de custo e de cobertura vegetal, mas fica fora das métricas de árvore.

2.4. Refinamento, indicadores e agregação

O refinamento padrão remove componentes conectados com menos de 64 pixels e preenche buracos internos menores que 64 pixels; abertura e fechamento morfológicos ficam desligados. Operações que adicionam área passam por uma trava: se a máscara crescer mais de 5% em relação à área bruta, o resultado volta ao estágio apenas subtrativo e recebe a *flag* `growth_guard`. O ajuste é puramente conservador: numa copa, unir regiões através do céu pode inventar folhas.

Por imagem, além de P_T , o pipeline calcula pixels válidos, pixels de árvore, cobertura vegetal e razões por grupo. Em *multi-view*, as razões individuais são resumidas por média, mediana, percentis 25 e 75, intervalo interquartil (IQR), desvio-padrão, mínimo e máximo, sem que nada seja somado entre vistas. A mediana amortece uma vista extrema e o IQR informa quanto o ponto depende do enquadramento. O total planejado e o total válido ficam ambos no resultado.

2.5. Interfaces, artefatos e reprodutibilidade

A CLI `tree-ai` analisa imagens, coordenadas ou endereços, avalia predições e valida exportações de anotação. A Web API em FastAPI oferece `/ping`, `/ready`, `/analyse/single` e `/analyse/multi`, devolvendo as métricas junto com backend, checkpoint e taxonomia; como não há autenticação e cada requisição pode acionar uma API paga, ela deve ficar em *localhost* ou atrás de proxy. Em cada execução é possível gravar manifesto, CSV, predições e, por vista, RGB, máscaras bruta e refinada, sobreposição e métricas. O manifesto registra versões, dispositivo, checkpoint, taxonomia e sementes; o JSON distingue máscara disponível, indisponível e omitida, e não serializa NaN nem infinito.

Boa parte do esforço de engenharia foi consumido por incompatibilidades que nenhum artigo costuma mencionar e que reaparecem em qualquer instalação nova. O Detectron2 importa `pkg_resources`, removido do `setuptools` na versão 81, o que quebra ambientes recentes até que se fixe `setuptools<81` em Linux, WSL e Windows por igual. O DeepLabV3+ de VainF é código de pesquisa sem `setup.py`, então `pip install -e` falha; em vez de exigir o clone, um script busca apenas o pacote `network` num commit fixado. Caminhos de checkpoint e de repositório são propriedade da máquina, não da execução, e por isso vivem no `.env`. Cada um desses casos está documentado em `docs/`, porque são exatamente os pontos onde um usuário final desiste antes de chegar à primeira medida.

Dependências de aprendizado de máquina só são importadas na construção de um backend, de modo que os testes rodam offline, sem GPU, sem baixar pesos e sem chave do Google. No estado auditado, 320 testes passaram em Python 3.13 com 86% de cobertura de ramos e instruções, e um segundo portão impede que qualquer módulo de produção caia abaixo de 60%. Hypothesis exercita RLE, máscara e cobertura, agregação e invariantes geográficas. Ruff, Black e Pyright passam sem erros. A CI cobre Python 3.10 e 3.13. Reproduzir os números exige fixar commit, checkpoints, versões, dispositivo, taxonomia, configuração de refinamento, anotações e imagens: a CLI embute esse manifesto em `predictions.json`, e execuções bem-sucedidas do DeepLab registram o SHA-256 do checkpoint. O notebook de figuras é a exceção conhecida; usa cache e registra dispositivo, mas seus arquivos intermediários de predição não recebem o manifesto completo, lacuna que se corrige chamando `build_manifest` e gravando o hash de todo checkpoint.

2.6. Conjunto rotulado e protocolo de anotação

O piloto usa sete quadros GSV de 600×400 pixels capturados em 2025, tomados em três pontos do ABC Paulista próximos aos campi da Universidade Federal do ABC. Quatro vêm da Alameda da Universidade, no bairro Anchieta, em São Bernardo do Campo (`id1`), com angulações horizontais de 0° , 90° , 180° e 270° . Os outros três vêm da Rua Abolição, na Vila São Pedro, em Santo André, nos números 53 (`id3`) e 340 (`id4`) da mesma via. Os pontos foram escolhidos à mão, de modo a varrer de ausência de árvores a copas dominantes, com um caso negativo incluído de propósito. Um quarto ponto, na Rua Arcturus (Jardim Antares, São Bernardo do Campo), foi levantado e descartado por representar condições próximas às já cobertas; ele responde pelo salto de `id1` para `id3` na numeração interna. O custo da anotação manual limitou o tamanho do conjunto: são sete imagens e três localizações, com forte dependência entre quatro delas.

As árvores foram anotadas manualmente no Roboflow e exportadas em COCO Instance Segmentation, um polígono por árvore distinguível; para a avaliação semântica, as instâncias de cada imagem foram unidas. O protocolo aceita copa e tronco visíveis, palmeiras e árvores em área privada visíveis da rua; exclui gramados, sebes, arbustos sem tronco claro, trepadeiras, sombras e folhas caídas; e não imagina pixels atrás de oclusores. Vegetação abaixo de cerca de 0,1% da imagem, ou indecível a duas vezes o zoom, fica de fora. Anotar por indivíduo e unir depois evita desenhar os mesmos pixels duas vezes, o que produziria dois *ground truths* discordantes.

Seis tarefas de rotulagem geraram JSON. O quadro sem árvores não gerou exportação no Roboflow, o único que testa falso positivo puro, e precisou ser reinserido à mão com zero anotações. IDs locais de imagem e anotação foram reenumerados, categorias unificadas pelo nome e o resultado validado por `tree-ai validate-dataset`. A primeira passagem de anotação levou cerca de duas horas; depois de fixado o protocolo do parágrafo anterior, as sete imagens foram redesenhadas do zero, o que consumiu mais uma a duas horas. São 41 árvores anotadas, cuja união preserva exatamente a área rotulada e mantém aberta a possibilidade de análises por indivíduo mais adiante.



Figura 2. Sete quadros do estudo e cobertura arbórea manual. Os quatro quadros id1 são ângulos diferentes de uma única localização.

2.7. Protocolo experimental e métricas

Todos os backends rodaram em modo *zero-shot*, sem qualquer ajuste nos sete quadros. OneFormer e Mask2Former usaram os checkpoints grandes ADE20K; o Detectron2, o modelo de zoo COCO Panoptic FPN R50 3x; o DeepLabV3+, o checkpoint Cityscapes MobileNet. Cada combinação imagem-backend foi medida com refinamento ligado e desligado, e a avaliação principal usa a máscara refinada. O cache em `measurements.csv` separa inferência de geração de gráficos, para que mudar o estilo de uma figura não reexecute os modelos.

Não há divisão treino/validação/teste porque nenhum parâmetro foi aprendido ou escolhido a partir das anotações. A inspeção dos resultados influencia interpretações,

então qualquer ajuste posterior de limiar, taxonomia ou morfologia exigirá conjunto de calibração separado e teste por localização. Diferentes *headings* de um mesmo ponto não podem ser repartidos entre treino e teste, por serem quase duplicatas espaciais.

O ambiente de testes foi um Windows 11 Pro Versão 10.0.22621, com Python 3.13, PyTorch 2.6.0+cu118, CUDA 12.8, Transformers 5.15.0 e Detectron2 0.6, em uma NVIDIA GeForce GTX 970. O tempo por quadro foi medido em torno de `analyse_image`, e a mediana reduz o efeito da primeira chamada, que inclui aquecimento e inicializações tardias em alguns backends. Como não houve sincronização explícita de eventos CUDA antes e depois de cada inferência, os tempos valem como custo operacional observado (um *benchmark* de kernels exigiria outra instrumentação).

No nível semântico, a predição binária é comparada à união manual no quadro completo, acumulando TP, FP, FN e TN. IoU, Dice, precisão e revocação são reportados em micro, isto é, com as contagens somadas sobre as sete imagens antes da divisão. Quando predição e *ground truth* são ambos vazios, a IoU individual fica indefinida e o quadro não recebe artificialmente 1,0 nem 0,0; a micro-IoU do conjunto continua bem definida. No nível do indicador, com $e_i = P_T(\hat{M}_i) - P_T(M_i)$, são reportados $MAE = n^{-1} \sum |e_i|$, $RMSE = \sqrt{n^{-1} \sum e_i^2}$ e o viés $n^{-1} \sum e_i$, todos em pontos percentuais, mais o pior caso absoluto. A correlação de Pearson aparece apenas como diagnóstico: uma predição igual ao dobro do valor verdadeiro teria correlação perfeita e erro inaceitável.

3. Resultados

3.1. Concordância com a anotação manual

A Tabela 2 traz a avaliação nas sete imagens. O OneFormer obteve IoU micro 0,880, Dice 0,936 e MAE de cobertura 0,963 p.p. O Mask2Former ficou próximo, com IoU 0,833 e MAE 1,456 p.p. O Detectron2 alcançou revocação 0,986 com precisão 0,721, padrão característico de supersegmentação; seu MAE e seu viés coincidem em 5,158 p.p. porque nenhum erro foi negativo, ou seja, em todos os quadros não vazios a cobertura foi superestimada. O DeepLab não recebeu números de árvore, pois o checkpoint Cityscapes não expressa a classe.

Tabela 2. Avaliação contra *ground truth* manual ($n = 7$ imagens, 41 árvores). Métricas de pixels são micro; erros de cobertura estão em pontos percentuais.

Backend	IoU	Dice	Prec.	Rev.	MAE	RMSE	Viés	Máx. abs.
OneFormer	0,880	0,936	0,935	0,937	0,963	1,267	0,017	2,712
Mask2Former	0,833	0,909	0,914	0,904	1,456	1,860	-0,156	3,434
Detectron2	0,714	0,833	0,721	0,986	5,158	7,348	5,158	16,820
DeepLabV3+	—	—	—	—	—	—	—	—

O quadro negativo foi previsto como 0,00% pelos três backends com classe de árvore. Um único negativo, sem vegetação ambígua, não caracteriza o comportamento de falso positivo do sistema. A média manual de cobertura foi 14,008%, contra 14,025% do OneFormer, 13,852% do Mask2Former e 19,166% do Detectron2, com correlações de 0,995, 0,990 e 0,955.

O resultado relevante para o objetivo do trabalho é que o pipeline reproduz a anotação manual dentro de cerca de um ponto percentual quando o backend tem a classe adequada, o que basta para tratar o número como medida operacional. A distância entre o primeiro e o segundo colocados, em sete quadros, não sustenta preferência entre eles.

3.2. Mesma arquitetura, espaços de classes diferentes

A separação entre modelo e taxonomia fica visível quando se mantém a arquitetura fixa e se troca só o checkpoint. O Mask2Former é o único backend publicado para vários espaços de classes, e a Tabela 3 mostra o quadro `id1_heading0` medido por todos eles.

Tabela 3. Um quadro (*ground truth* manual 29,26%), cinco configurações. A troca de espaço de classes, com arquitetura idêntica, retira o indicador em vez de reduzi-lo.

Backend	Checkpoint	Árvore (%)	Vegetação (%)
OneFormer	ADE20K, Swin-L	31,97	42,68
Mask2Former	ADE20K, Swin-L	32,69	42,88
Mask2Former	Cityscapes, Swin-T	indisponível	36,24
Detectron2	COCO-panoptic, R50	36,21	46,00
DeepLabV3+	Cityscapes, MobileNet	indisponível	34,54

A terceira linha é a que interessa. A mesma arquitetura que produz 32,69% com pesos ADE20K devolve `tree_source=unavailable` com pesos Cityscapes, em vez de entregar discretamente o número de vegetação. Sem esse comportamento no núcleo, a diferença entre 32,69% e 36,24% pareceria desacordo entre modelos, quando na verdade um deles não tem a classe.

3.3. Desacordo entre backends e mecanismo do erro

A Figura 3 mostra cada imagem, os três resultados com classe de árvore e o diamante do *ground truth*. O intervalo médio entre menor e maior backend foi 5,47 p.p., com máximo de 17,60 p.p. em `id3_heading165`, onde o Detectron2 previu 40,20% para um valor manual de 23,38%. A classe `tree-merged` do COCO capturou quase todos os pixels rotulados e ainda incluiu regiões vizinhas, combinação que explica a alta revocação com precisão baixa. Diferença de taxonomia e o limiar conservador do protocolo manual são explicações compatíveis com o padrão; sete quadros não permitem isolar uma delas.

O erro máximo do OneFormer foi 2,71 p.p.; o do Mask2Former, 3,43 p.p. Nos dois casos os sinais alternam entre quadros, o que deixa o viés perto de zero. Vale registrar que viés médio nulo convive com erros grandes que se cancelam, e é por isso que a tabela traz também RMSE e pior caso.

3.4. Single-view e multi-view

Nas quatro angulações do mesmo ponto, o *ground truth* variou de 1,17% a 29,26%, uma amplitude de 28,09 p.p., com mediana manual de 17,25% e IQR de 10,63 p.p. Essa variação é propriedade da cena: as quatro imagens contêm mesmo proporções visíveis diferentes. Uma vista isolada, portanto, só representa a localização se o protocolo fixar direção e campo de visão.

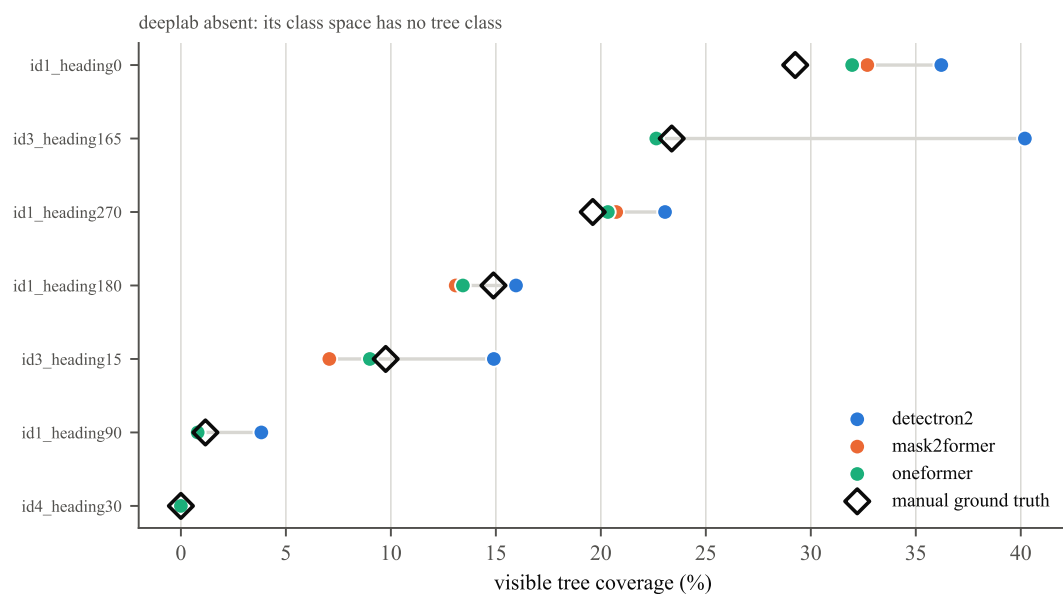


Figura 3. Cobertura por imagem. O segmento cinza mostra o desacordo entre backends e o diamante preto, a anotação manual. DeepLab está ausente por não possuir classe de árvore.

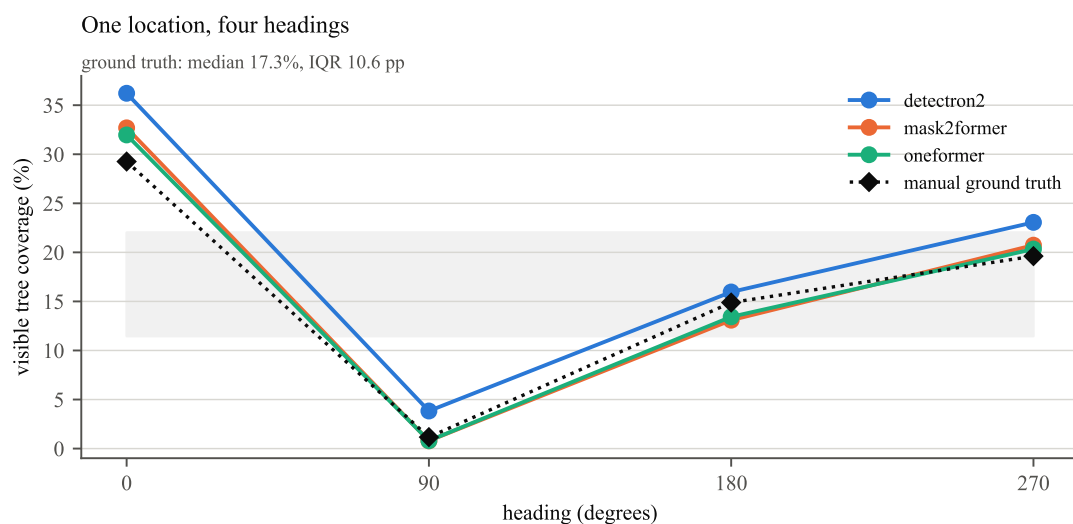


Figura 4. Um ponto, quatro angulações. A faixa cinza representa o IQR do *ground truth*. A mudança de direção domina a diferença entre OneFormer e Mask2Former.

As medianas dos quatro quadros foram 16,88% (OneFormer), 16,91% (Mask2Former) e 19,51% (Detectron2), com erros de mediana de $-0,37$, $-0,34$ e $+2,26$ p.p., e IQR entre 12,97 e 13,71 p.p. O multi-view amostra mais direções e expõe a dispersão pelo IQR. Ele não corrige oclusão, não reconstrói árvores invisíveis e não soma árvores entre vistas: a mesma copa pode aparecer repetidamente, já que o sistema não faz associação geométrica entre angulações. Para medir uma rua com alguma precisão, o ponto de captura precisa vir de uma regra independente das árvores, de preferência sobre o eixo ou o trajeto dos pedestres de interesse, com angulação e campo de visão fixados antes da inferência.

3.5. Refinamento e análise qualitativa

O refinamento padrão alterou no máximo 0,027 p.p. em todas as combinações de imagem e *backend*, e a trava de crescimento nunca foi acionada. Os resultados principais vêm, então, do modelo e da taxonomia. O conjunto também não testa situações em que a trava seria decisiva, e uma expansão do estudo deveria escolher a configuração de refinamento em validação separada, com casos de copas fragmentadas.

A Figura 5 traz o pior e o melhor quadro pela média de IoU, selecionados pelo número em vez de a olho. O pior, `id1_heading90`, tem apenas 1,17% de *ground truth*: grupos pequenos de pixels causam grande mudança relativa, e o Detectron2 incorpora vegetação distante na encosta, caindo para IoU 0,25. OneFormer e Mask2Former mantêm cobertura próxima da manual com contornos que não coincidem inteiramente. No melhor quadro, `id1_heading180`, os três acompanham as copas principais, e o Detectron2 chega a IoU 0,92 apesar de cobertura maior, o que demonstra que erro de área e erro espacial andam juntos sem serem a mesma coisa.

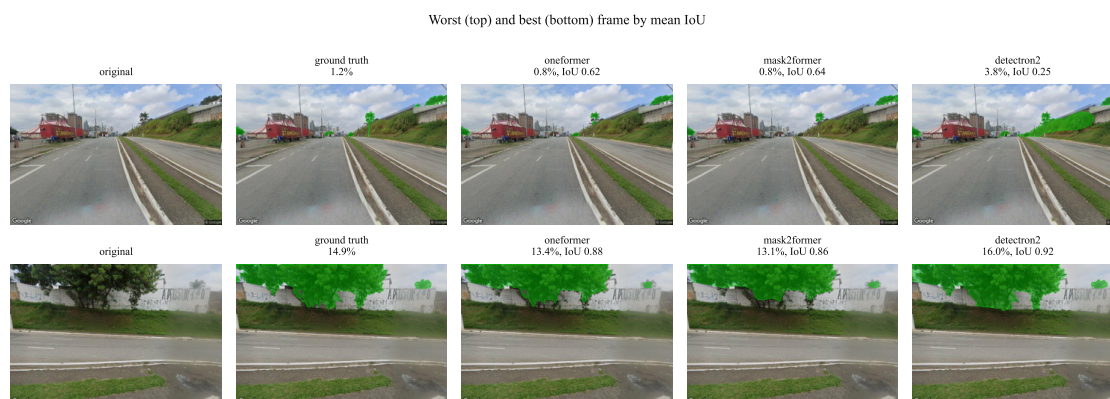


Figura 5. Pior (acima) e melhor (abaixo) quadro pela IoU média. Verde indica máscara sobreposta à imagem original.

3.6. Custo observado

Na GTX 970, as medianas por quadro foram 0,064 s para o DeepLabV3+, 0,230 s para o Mask2Former, 0,232 s para o Detectron2 e 1,148 s para o OneFormer. O DeepLab é rápido e responde à pergunta mais ampla de vegetação. Entre os backends que expressam árvore, o Mask2Former junta tempo baixo e o segundo melhor erro, enquanto o OneFormer custa cerca de cinco vezes mais. A comparação exclui download de pesos, requisição ao GSV, gravação de artefatos e concorrência no serviço.

3.7. O que os resultados sustentam e o que não sustentam

Dentro desta amostra, o OneFormer aproximou-se mais da anotação manual, seguido pelo Mask2Former, e o Detectron2 preservou quase toda a área rotulada ampliando sistematicamente a classe. Esses números sustentam, antes de tudo, que o pipeline funciona: um usuário que rode o `tree-ai` com um backend adequado obtém, nestes quadros, um valor a cerca de um ponto percentual da anotação manual, com o caminho inteiro registrado em manifesto. Sustentam também que taxonomia e checkpoint movem o indicador, e que nenhuma arquitetura pode ser avaliada sem que se diga qual espaço de classes foi usado: uma diferença de 17,60 p.p. entre backends num único quadro é grande o bastante para mudar uma decisão urbana, caso o número circule sem essa informação.

O conjunto não sustenta generalização. Sete imagens dão estimativas instáveis, quatro compartilham a localização, duas das três localizações estão na mesma via, existe um único negativo e todos os quadros têm a mesma resolução e a mesma fonte, sem estações, chuva, noite, oclusão veicular forte ou diversidade de bairros. Os 1,68 milhão de pixels tampouco são 1,68 milhão de observações independentes, já que pixels vizinhos e vistas de um mesmo ponto são correlacionados; intervalos de confiança calculados por pixel sairiam artificialmente estreitos. A unidade de reamostragem adequada é a localização ou o segmento de rua, e destes há poucos aqui.

A contagem de pixels, por fim, é a definição do indicador. Problemas de leitura aparecem quando P_T é tratado como coisa diversa: uma rua com muitas árvores pequenas ou distantes pode ter cobertura visual baixa, enquanto uma única copa próxima domina o quadro; ônibus, fachadas e enquadramento mexem no numerador, e a perspectiva faz o que está perto ocupar mais área. O indicador serve como exposição visual, ou ocupação da cena, e não substitui inventário, densidade de indivíduos, área basal ou projeção aérea de copa. Um protocolo de cidade precisa definir espaçamento entre pontos, lado ou eixo de captura, altura aproximada, angulação horizontal, inclinação e campo de visão, e a agregação deve resumir primeiro as angulações de captura de cada ponto e só depois os pontos de cada segmento, para que uma localização com mais vistas não pese mais que as outras.

Duas dependências externas completam o quadro. O GSV oferece cobertura e padronização em troca de dependência temporal e contratual: a imagem pode ter meses ou anos, representar outra estação e não refletir poda, remoção ou plantio recente [Google 2026], o panorama pode ser substituído e nem toda via está disponível, de modo que todo resultado deveria vir acompanhado da data do panorama, e a redistribuição de quadros precisa respeitar os termos do provedor. Os pesos, por sua vez, vêm de ADE20K, COCO e Cityscapes, nenhum deles construído sobre arborização urbana brasileira. O Cityscapes cobre 50 cidades europeias em condições diurnas e clima bom ou moderado [Cordts et al. 2016] e o ADE20K é mais amplo sem ter sido pensado para a taxonomia fina de árvores daqui [Zhou et al. 2017]. Segmentadores viários perdem desempenho ao atravessar cidades e continentes [Chen et al. 2017], e o bom desempenho nos sete quadros locais é compatível tanto com robustez quanto com sorte amostral.

4. Lições Aprendidas

O que funcionou melhor foi tornar explícitos os contratos que costumam ficar implícitos. A taxonomia impede confundir ausência de classe com zero de árvore; `tree_source`

distingue classe real, proxy e indisponibilidade; o plano de vistas evita seleção orientada pela própria máscara; e separar IoU de MAE mostrou que o indicador pode estar numericamente certo com contorno imperfeito. Manter aquisição, medida e exportação num único núcleo também barateou cada backend novo: o adaptador do Mask2Former custou pouco mais que declarar sua taxonomia.

O caminho para métricas por árvore foi o que não deu certo. Os checkpoints disponíveis não fornecem árvores individuais: no COCO-panoptic, *tree-merged* é *stuff*; no ADE20K utilizado, árvore também é tratada semanticamente; no Cityscapes, a classe ampla é vegetação. O COCO-80 traz apenas *potted plant*, as 1.203 categorias do LVIS v1 contêm somente *Christmas_tree*, e os modelos de instância de árvore que se pode baixar hoje — *detectree2*, *DeepForest*, *restor/tcd-mask-rcnn-r50* — são treinados em imagens aéreas, onde as copas aparecem separadas; da rua elas se sobrepõem e se ocluem. Heurísticas de componentes conectados separariam ou fundiriam copas conforme o acaso dos contatos, sem identidade semântica, e por isso AP, recall por indivíduo e contagens foram removidos em vez de permanecerem como caminho de código inalcançável. Nada disso torna a tarefa impossível: há trabalhos que detectam árvores em panoramas e combinam vistas para geocodificar indivíduos [Laumer et al. 2020], e pipelines específicos treinam detectores para espécies e perfis urbanos [Choi et al. 2022]. A limitação deste projeto foi estreita porque não existia checkpoint *tree-as-thing* plug-and-play compatível com os backends, e 41 instâncias não validam um modelo individual generalizável.

O DeepLab ensinou algo próximo: uma arquitetura rápida e competente pode ser inadequada porque seu espaço de classes não responde à pergunta, e devolver “indisponível” informa mais que devolver zero. A rotulagem trouxe outros dois detalhes capazes de alterar a avaliação em silêncio. Não foi possível exportar, via Roboflow, o quadro sem árvores, justamente o único que testa falso positivo puro, e ele teve de ser reinserido à mão. E seis exports de uma mesma imagem não podem ser avaliados em separado e depois promediados como se micro-IoU fosse média de IoUs.

Com mais tempo, três coisas seriam feitas em outra ordem. O protocolo de anotação ficou estável somente após da primeira passagem inteira, e refazer as sete imagens custou quase o mesmo que anotá-las na primeira vez. Portanto, escrever as regras antes de abrir o Roboflow teria saído mais barato. A configuração de refinamento foi escolhida por bom senso, quando o certo seria reunir antes uma amostra de validação com falhas morfológicas e copas fragmentadas. E a unidade de inferência deveria ter sido fixada junto com o desenho amostral, antes de coletar quadros convenientes. O software suporta a hierarquia, mas o conjunto piloto não chega à análise de cidade.

O passo seguinte, portanto, é menos sobre arquitetura e mais sobre amostra: ampliar o *ground truth* por amostragem estratificada de localizações brasileiras, estações, morfologias viárias e níveis de arborização, preservando negativos e separando treino, validação e teste por segmento; comparar zero-shot, adaptação de domínio e *fine-tuning* com intervalos reamostrados por localização; e estudar quantas direções bastam para estabilizar mediana e IQR. Para indivíduos, o caminho correto é formular outro alvo, com detector *tree-as-thing* próprio e avaliação separada da cobertura. Do lado da ferramenta, as extensões úteis são batch geográfico, autenticação e limite de custo na API, versionamento de panoramas e comparação com inventário de campo e cobertura aérea.

Referências

- Cai, B. Y., Li, X., Seiferling, I., and Ratti, C. (2018). Treepedia 2.0: Applying deep learning for large-scale quantification of urban tree cover. In *2018 IEEE International Congress on Big Data*, pages 49–56.
- Carvalho, J. O. (2026a). Sidewalk analysis. GitHub repository.
- Carvalho, J. O. (2026b). Urban tree coverage: Visible street-level tree coverage from street view imagery using semantic segmentation. GitHub repository, commit e411123.
- Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., and Adam, H. (2018). Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proceedings of the European Conference on Computer Vision*, pages 801–818.
- Chen, Y.-H., Chen, W.-Y., Chen, Y.-T., Tsai, B.-C., Wang, Y.-C. F., and Sun, M. (2017). No more discrimination: Cross city adaptation of road scene segmenters. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1992–2001.
- Cheng, B., Misra, I., Schwing, A. G., Kirillov, A., and Girdhar, R. (2022). Masked-attention mask transformer for universal image segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1290–1299.
- Choi, K. et al. (2022). An automatic approach for tree species detection and profile estimation of urban street trees using deep learning and google street view images. *ISPRS Journal of Photogrammetry and Remote Sensing*, 190:165–180.
- Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., and Schiele, B. (2016). The Cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3213–3223.
- Google (2026). Google-contributed street view imagery policy. <https://www.google.com/streetview/policy/>. Acesso em: 19 ago. 2026.
- Google Maps Platform (2026). Street view image metadata. <https://developers.google.com/maps/documentation/streetview/metadata>. Acesso em: 19 ago. 2026.
- Jain, J., Li, J., Chiu, M. T., Hassani, A., Orlov, N., and Shi, H. (2023). OneFormer: One transformer to rule universal image segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2989–2998.
- Kirillov, A., Girshick, R., He, K., and Dollár, P. (2019a). Panoptic feature pyramid networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6399–6408.
- Kirillov, A., He, K., Girshick, R., Rother, C., and Dollár, P. (2019b). Panoptic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9404–9413.
- Laumer, D., Lang, N., van Doorn, N., Mac Aodha, O., Perona, P., and Wegner, J. D. (2020). Geocoding of trees from street addresses and street-level images. *ISPRS Journal of Photogrammetry and Remote Sensing*, 162:125–136.

- Li, X., Zhang, C., Li, W., Ricard, R., Meng, Q., and Zhang, W. (2015). Assessing street-level urban greenery using Google Street View and a modified green view index. *Urban Forestry & Urban Greening*, 14(3):675–685.
- Seiferling, I., Naik, N., Ratti, C., and Proulx, R. (2017). Green streets: Quantifying and mapping urban trees with street-level imagery and computer vision. *Landscape and Urban Planning*, 165:93–101.
- Zhou, B., Zhao, H., Puig, X., Fidler, S., Barriuso, A., and Torralba, A. (2017). Scene parsing through ADE20K dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 633–641.